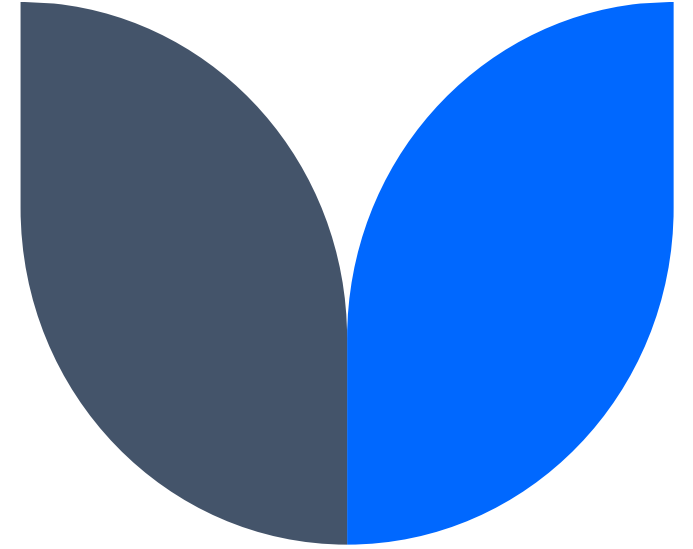
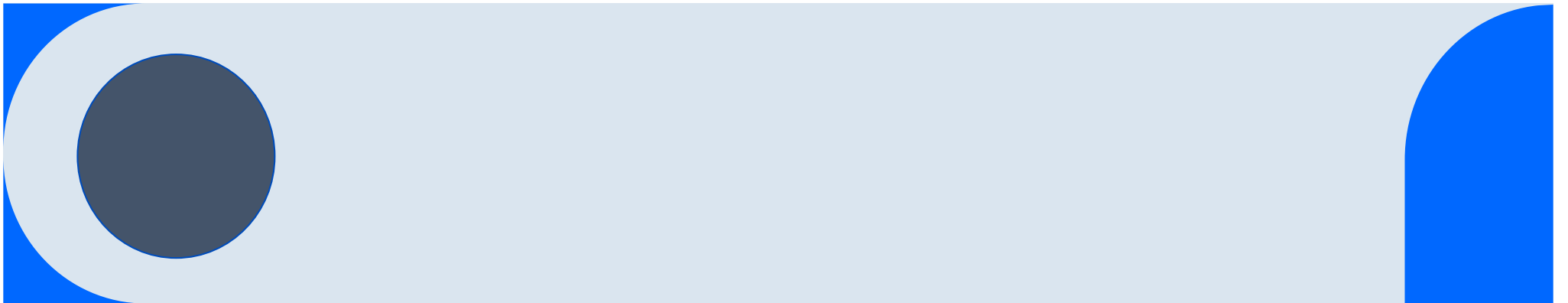




Angular





Índice

- Introducción
 - Contexto
 - Funcionalidades básicas
 - Routing
 - Directivas, control de flujo y tuberías
 - Modelos
 - Paso de parámetros
 - Servicios y peticiones HTTP
 - Formularios
 - Otros
- 

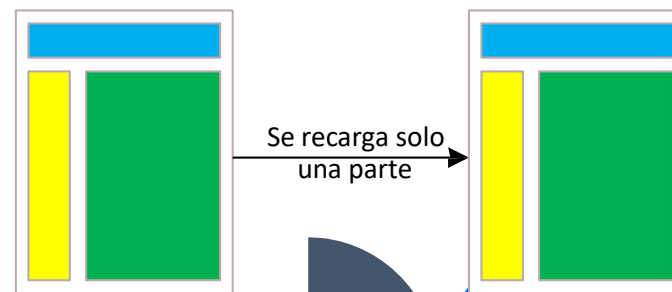
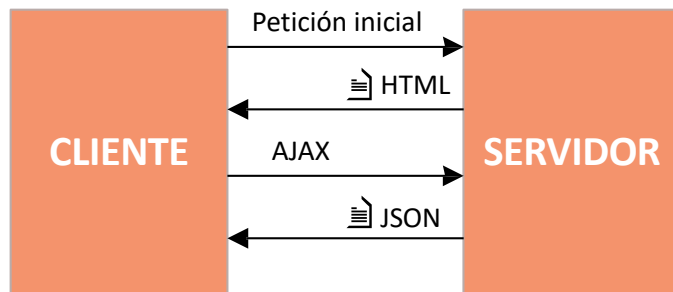


Índice

- Contexto
- Funcionalidades básicas
- Routing
- Directivas, control de flujo y tuberías
- Modelos
- Paso de parámetros
- Servicios y peticiones HTTP
- Formularios
- Otros

Descripción

- Angular es una plataforma de código libre para el desarrollo de aplicaciones web desarrollada por Google
- Utiliza HTML y TypeScript
- También permite el desarrollo de apps móviles
- Permite la creación de aplicaciones web de tipo **SPA** (*Single Page Application*):



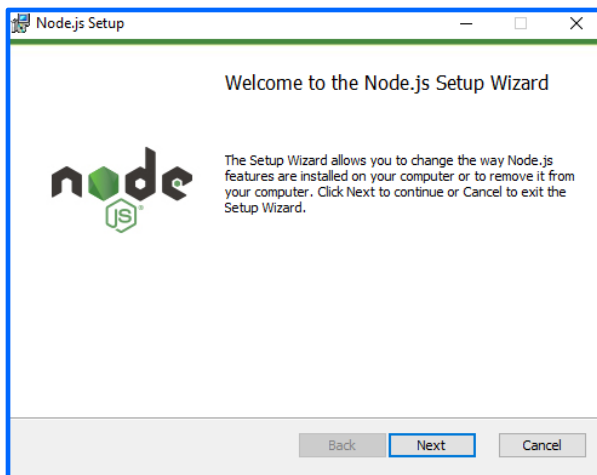
Un poco de historia...

- AngularJS 1:
 - Basado en JavaScript
 - Lanzamiento inicial: 2010
 - Última versión: 2020
 - Desde 2022 Google no mantiene AngularJS 1 → obsoleto
- Angular:
 - Basado en TypeScript
 - No compatible con AngularJS 1
 - Lanzamiento inicial: 2016 → inicialmente se llamó Angular 2
 - Desde 2016 se han ido sucediendo diversas versiones (hasta 2 versiones por año)
 - Última versión: Junio 2024 (versión 18)

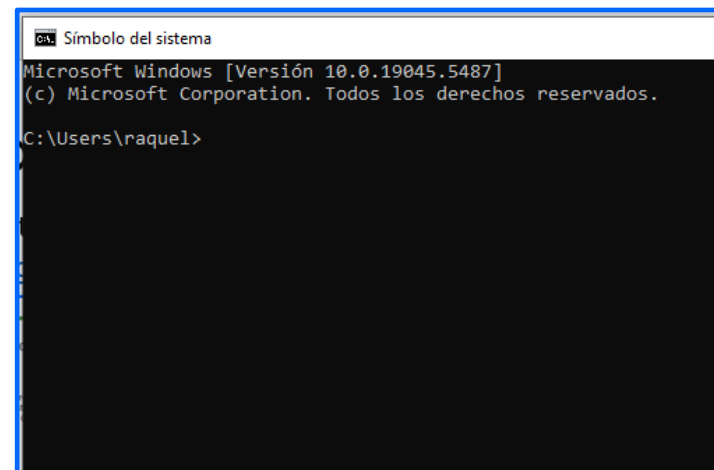


Instalación

1. Instalar la última versión de Node.js:
<https://nodejs.org>



2. Abrimos la línea de comandos (cmd en Windows, terminal en Mac y Linux)



3. Actualizamos NPM, el gestor de paquetes de Node.js:

```
>> npm install -g npm@latest
```

Instalación

4. Eliminamos versiones antiguas de Angular CLI (por si lo teníamos instalado previamente)

```
>> npm uninstall -g angular-cli  
>> npm uninstall -g @angular/cli
```

5. Borramos la caché de Node.js:

```
>> npm cache clean --force
```

6. Instalamos la última versión de Angular CLI

```
>> npm install -g @angular/cli@latest
```

(Ver versión instalada)

```
>> ng version
```



```
Administrador: C:\WINDOWS\system32\cmd.exe  
  
D:\>ng version  
  
Angular CLI  
  
Angular CLI: 19.1.8  
Node: 22.14.0  
Package Manager: npm 11.1.0  
OS: win32 x64  
  
Angular:  
...  


| Package                    | Version             |
|----------------------------|---------------------|
| @angular-devkit/architect  | 0.1901.8 (cli-only) |
| @angular-devkit/core       | 19.1.8 (cli-only)   |
| @angular-devkit/schematics | 19.1.8 (cli-only)   |
| @schematics/angular        | 19.1.8 (cli-only)   |


```



Pregunta

Indica la afirmación
correcta

- A. Angular es compatible con TypeScript pero no con JavaScript
- B. Angular es compatible con JavaScript pero no con TypeScript
- C. Angular es compatible con TypeScript y con JavaScript
- D. Todas las respuestas anteriores son falsas

Pregunta

Indica la afirmación correcta

- A. Angular es compatible con TypeScript pero no con JavaScript
- B. Angular es compatible con JavaScript pero no con TypeScript
- C. Angular es compatible con TypeScript y con JavaScript
- D. Todas las respuestas anteriores son falsas

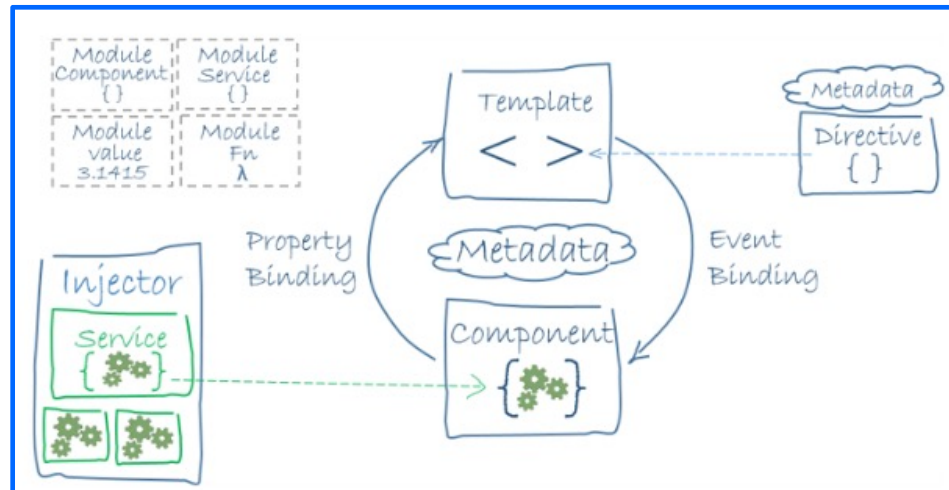


Índice

- Introducción
- Contexto
 - Arquitectura
 - Creación de un nuevo proyecto
 - Estructura de un proyecto
- Funcionalidades básicas
- Routing
- Directivas, control de flujo y tuberías
- Modelos
- Paso de parámetros
- Servicios y peticiones HTTP
- Formularios
- Otros

Arquitectura (no standalone)

- Módulo
- Decorador
- Directiva
- Componente
- Plantilla
- Tubería
- Metadatos
- Modelo
- *Data binding*
- Servicio
- Inyección de dependencias



<https://docs.angular.lat/guide/architecture>



Arquitectura

- **Módulo:** recopila bloque de código dedicado a un solo propósito
 - El uso de módulos ayuda a gestionar el desarrollo de aplicaciones complejas y su reutilización
 - Angular se envía como una colección de módulos JavaScript (también llamados librerías)....
 - Con la utilización de módulos se permite la carga en diferido, que minimiza la cantidad de código que se carga al inicio → mejora la experiencia de los usuarios
 - Las aplicaciones no standalone tienen un módulo raíz llamado AppModule, que proporciona el mecanismo de arranque que inicia la aplicación



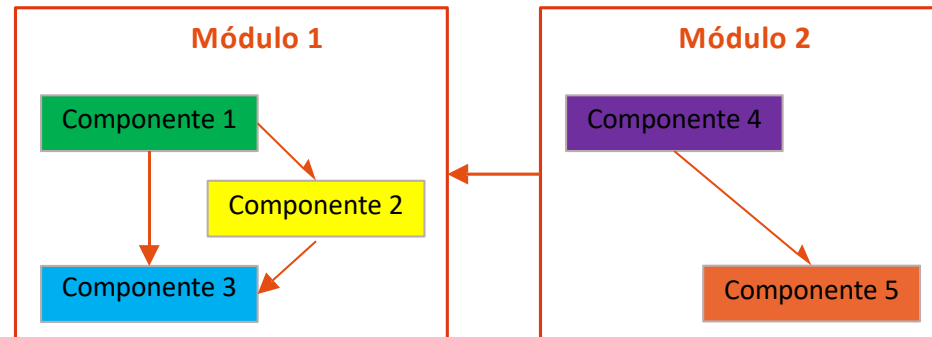
Arquitectura

- **Módulo:**
 - Angular utiliza módulos JavaScript estándar y también define un módulo Angular, `NgModule`
 - Una aplicación de Angular se define por un conjunto de `NgModules`
 - Los `NgModules` recopilan código relacionado en conjuntos funcionales
 - Los `NgModules` pueden importar la funcionalidad de otros, y permiten que su propia funcionalidad sea exportada y utilizada por otros `NgModules`
 - Un `NgModule` declara un contexto de compilación para un conjunto de componentes dedicados a un dominio de aplicación, un flujo de trabajo o un conjunto de capacidades estrechamente relacionadas

Arquitectura

Módulo

- Las principales propiedades que contiene son:
 - Declarations: vistas que pertenecen al módulo
 - Exports: declaraciones visibles para componentes de otros módulos
 - Imports: clases de otros módulos que se pueden utilizar en el módulo actual
 - Providers: servicios utilizados globalmente y accesibles desde cualquier parte de la aplicación
 - Bootstrap: vista principal o componente raíz, que aloja el resto de vistas de la aplicación
- Un módulo puede agrupar diversos componentes relacionados entre sí:



- Un componente puede no pertenecer a ningún módulo (*standalone*)



Arquitectura

- **Decorador:** función que modifica una definición de clase o propiedad
 - Decorador de clase: declara que la clase es del tipo dado y proporciona metadatos adecuados para el tipo. Tipos:
 - [@NgModule](#)
 - [@Directive](#)
 - [@Component](#)
 - [@Pipe](#)
 - [@Injectable](#)
 - Decorador de campo de clase: se declara inmediatamente antes de un campo en una definición de clase. Tipos:
 - [@Input](#)
 - [@Output](#)

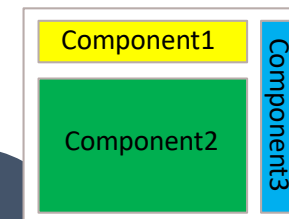


Arquitectura

- **Directivas:** clases que pueden modificar la estructura o los atributos en el DOM
 - Uso decorador [`@Directive\(\)`](#)
 - Antes de mostrar una vista, Angular evalúa las directivas y resuelve la sintaxis de enlace en la plantilla para modificar los elementos HTML y el DOM, de acuerdo con los datos y la lógica del programa
 - Hay tres categorías de directivas:
 - Componentes: asocia una plantilla con una clase
 - Directivas de atributos: modifica el comportamiento y apariencia de los elementos de una página (e.g. `ngStyle`, `ngClass`...)
 - Directivas estructurales: modifican la estructura del DOM (e.g. `ngFor`, `ngIf`...)

Arquitectura

- **Componente:**
 - Los componentes son los bloques básicos que componen una aplicación
 - Los componentes definen vistas, que son conjuntos de elementos de la pantalla que Angular puede elegir y modificar de acuerdo con la lógica y los datos de los programas
 - Los componentes usan servicios, los cuales proporcionan una funcionalidad específica que no está directamente relacionada con las vistas...
 - Cada componente define una clase que contiene datos y lógica de la aplicación, y está asociado con una plantilla HTML que define una vista que se mostrará en un entorno de destino
 - Componente = plantilla + metadatos + clase
 - Uso decorador [`@Component\(\)`](#)





Arquitectura

- **Plantilla (*template*)**: código que define cómo representar la vista de un componente
 - Es un fragmento de HTML
 - Una plantilla HTML Angular muestra una vista en el navegador, como HTML normal, pero con mucha más funcionalidad
- **Tubería (*pipe*)**: define una función que transforma los valores de entrada en valores de salida para mostrar en una vista
 - Angular define varias tuberías (por ejemplo, para cambiar a mayúsculas o para representar fechas)
 - Los usuarios pueden definir nuevas → uso de decorador [@pipe\(\)](#)



Arquitectura

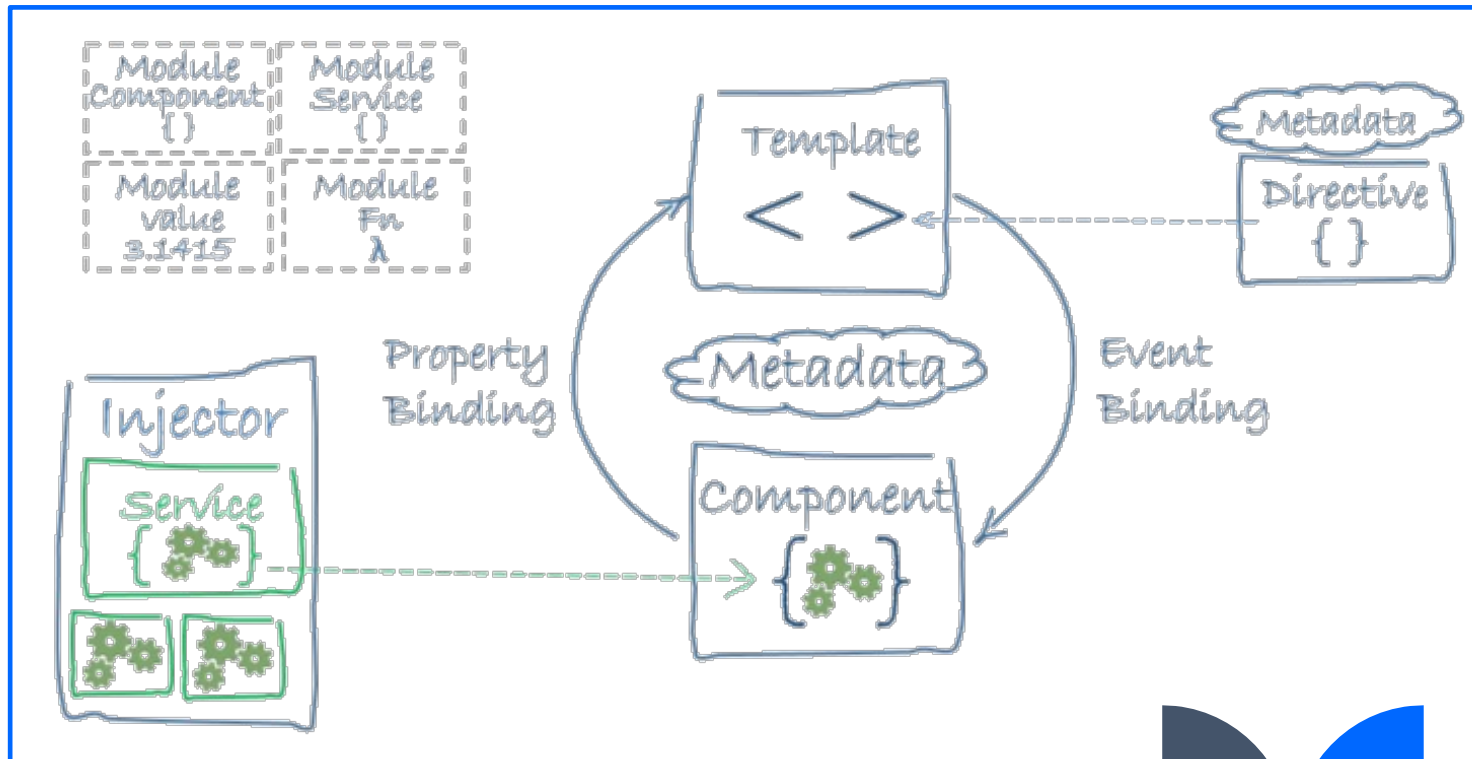
- **Metadatos:** permiten decorar una clase y configurar su comportamiento
- **Modelo:** clase que representa un objeto con propiedades que permite la utilización de la misma desde clases externas, evitando la reiteración de código
- ***Data binding*:** permite intercambiar datos entre la plantilla y la clase del componente que soporta la lógica del mismo. Existen 4 tipos de *Data binding*:
 - Interpolación: muestra el valor de una propiedad en el lugar donde se incluye → e.g. {{propiedad}}
 - Property binding: traspaso del objeto del componente padre al componente hijo → e.g. [objHijo] = "objPadre"
 - Event binding: invocación a un método del componente cuando se produce un evento → e.g. (click) = funcion(obj)
 - Two-way binding: binding bidireccional que permite combinar *event* y *property binding* → e.g. (ngModel) = "obj"



Arquitectura

- **Servicio:** clase que permite realizar acciones concretas que pone a disposición de cualquier componente
 - Ejemplo: permiten obtener datos, resolver lógicas especiales o realizar peticiones a un servidor...
- **Inyección de dependencias:** mecanismo para crear y entregar algunas partes de una aplicación (funcionalidades, servicios, etc.) a otras partes de la aplicación
 - Utiliza decorador [@Injectable](#)
 - Generalmente proporciona servicios (también pueden ser valores, como cadenas o funciones)
 - Facilita los test y modularidad del código

Arquitectura (no standalone)





Pregunta

¿Qué está compuesto por una clase, una plantilla y metadatos?

- A. Componente
- B. Decorador
- C. Servicio
- D. Directiva



Pregunta

¿Qué está compuesto por una clase, una plantilla y metadatos?

- A. **Componente**
- B. Decorador
- C. Servicio
- D. Directiva



Pregunta

¿Qué elemento
representa cada una de
las vistas de la
aplicación?

- A. Componente
- B. Decorador
- C. Servicio
- D. Directiva



Pregunta

¿Qué elemento
representa cada una de
las vistas de la
aplicación?

- A. **Componente**
- B. Decorador
- C. Servicio
- D. Directiva

Creación de nuevo proyecto en Angular

- Hasta Angular 16 los proyectos por defecto se creaban del tipo “no standalone”, en el que cada componente pertenece a un NgModule que le proporciona las dependencias
- Desde Angular 17, los proyectos por defecto son “standalone”, en los que los componentes son independientes y no tienen por qué pertenecer a un módulo
- Si queremos crear proyectos con configuración “no standalone” hay que indicar:

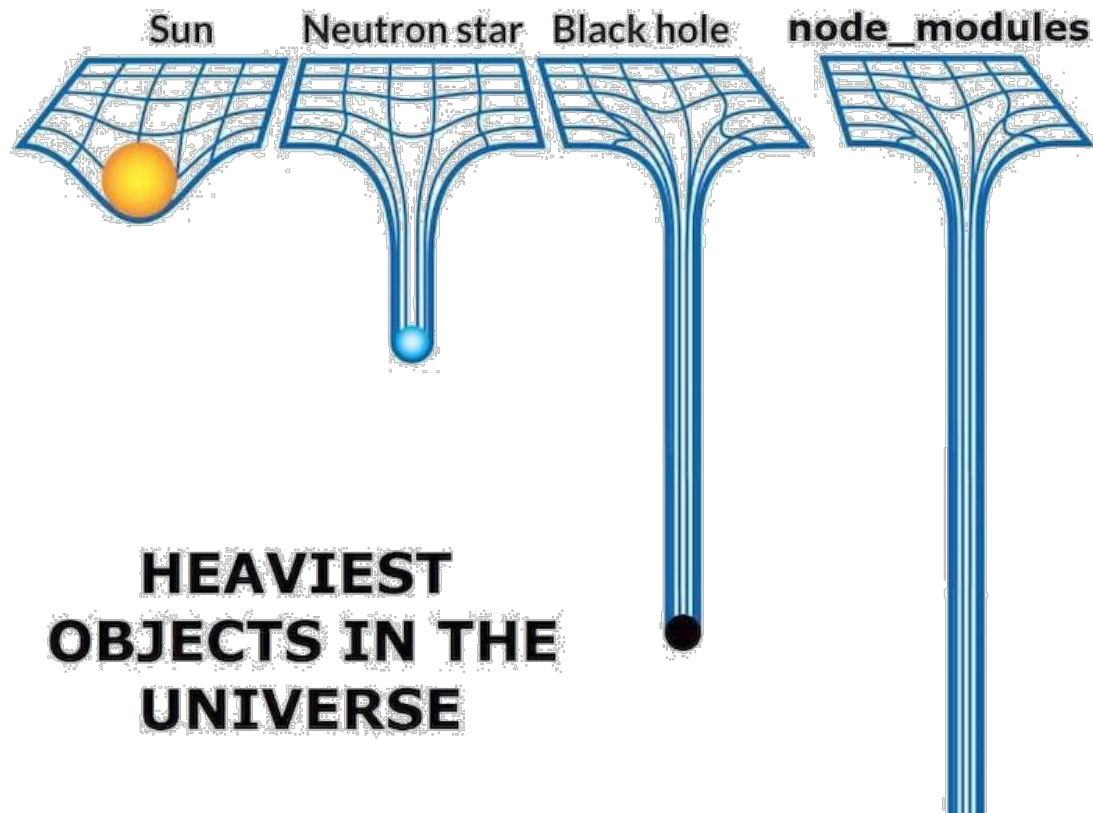
```
>> ng new --no-standalone
```

Creación de nuevo proyecto en Angular

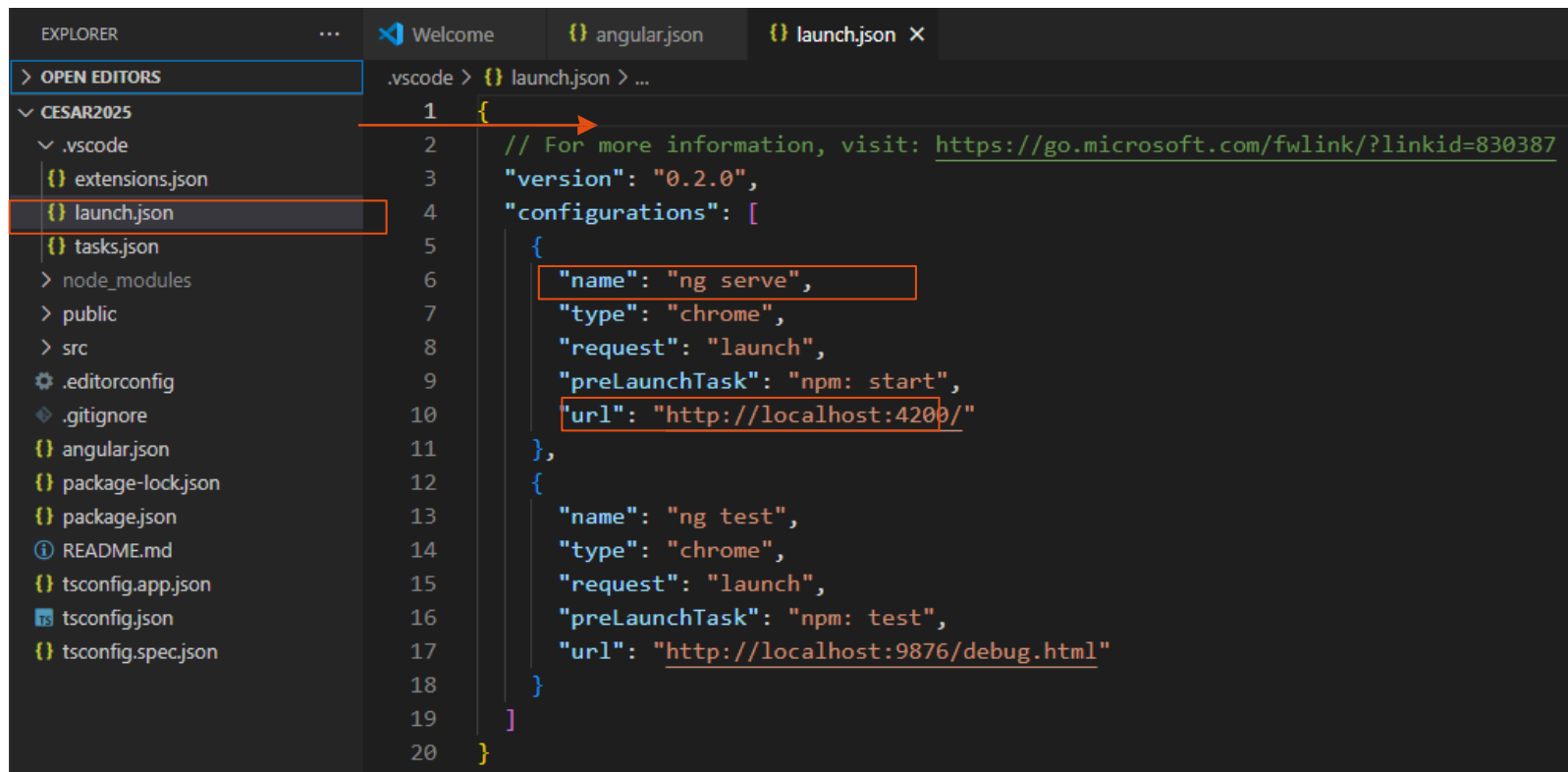
Lo primero: ¿qué nombre le ponemos a nuestro proyecto?



Creación de nuevo proyecto en Angular



Estructura de un proyecto en Angular



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure for 'CESAR2025'. The file 'launch.json' is selected and highlighted with an orange box. The main editor area shows the content of 'launch.json', which is a JSON configuration for launching the application. The configuration includes a version and a list of configurations for 'ng serve' and 'ng test'. The 'ng serve' configuration is also highlighted with an orange box. A red arrow points from the 'launch.json' file in the Explorer to the first line of the JSON object in the editor.

```
1 {
2   // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
3   "version": "0.2.0",
4   "configurations": [
5     {
6       "name": "ng serve",
7       "type": "chrome",
8       "request": "launch",
9       "preLaunchTask": "npm: start",
10      "url": "http://localhost:4200/"
11    },
12    {
13      "name": "ng test",
14      "type": "chrome",
15      "request": "launch",
16      "preLaunchTask": "npm: test",
17      "url": "http://localhost:9876/debug.html"
18    }
19  ]
20 }
```

Configuración de la información del servidor local

Arranque del servidor

- Accedemos a la ruta donde está creado nuestro proyecto Angular y arrancamos el servidor:

```
C:\Users\raquel\developer\angular\cesar2025>ng serve
```

```
npm
ng analytics disable Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.
Global setting: enabled
Local setting: enabled
Effective status: enabled
Browser bundles
Initial chunk files | Names          | Raw size
polyfills.js       | polyfills      | 90.20 kB | e: https://github.com/angular/angular-cli/issues
main.js            | main           | 23.23 kB |
styles.css         | styles         | 95 bytes |
                   | Initial total  | 113.53 kB

Server bundles
Initial chunk files | Names          | Raw size
polyfills.server.mjs | polyfills.server | 572.91 kB |
main.server.mjs     | main.server     | 24.55 kB |
server.mjs          | server          | 1.86 kB |

Application bundle generation complete. [13.655 seconds]

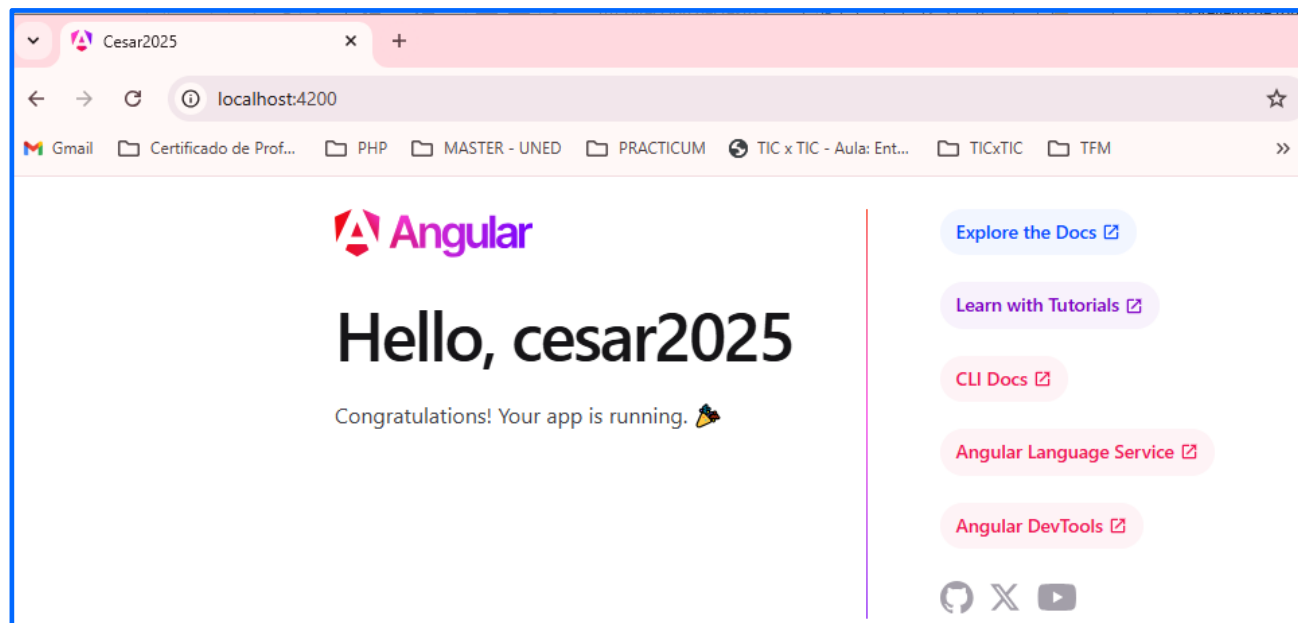
Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  Local: http://localhost:4200/
  press h + enter to show help
```

NOTA: Para parar el servidor utilizaremos Ctrl+C

Acceso a nuestra página

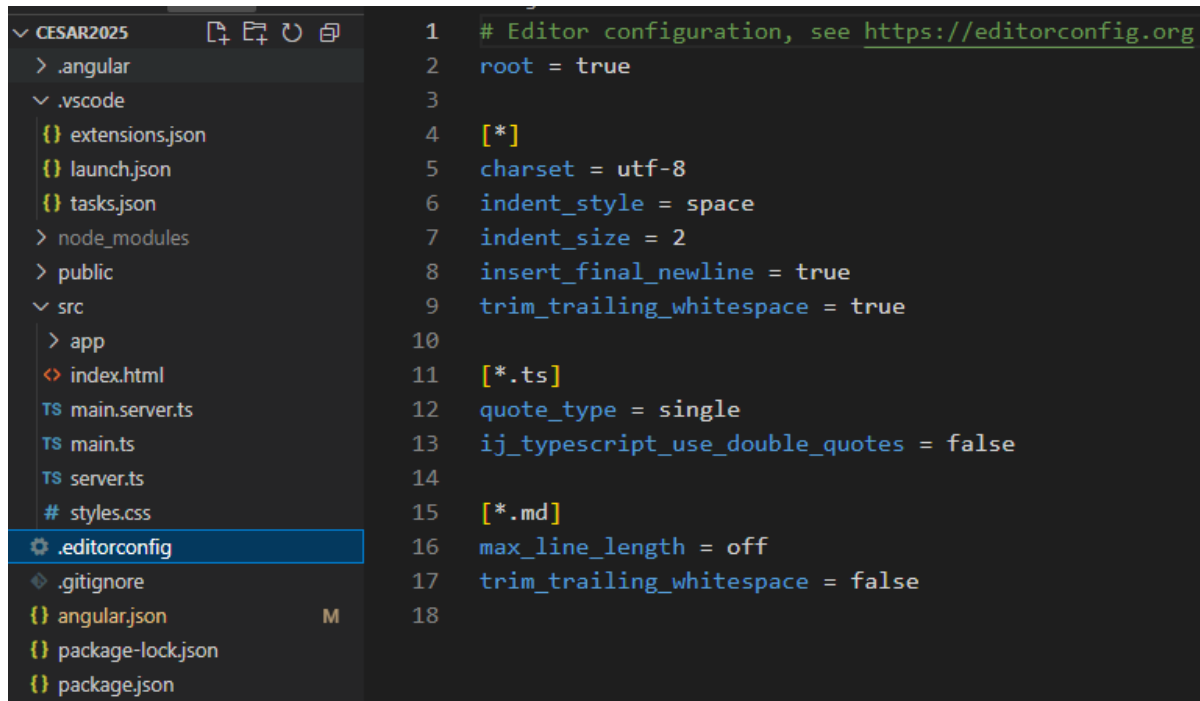
- Desde un navegador accedemos a nuestra página a través de la URL:

<http://localhost:4200>



Estructura de un proyecto Angular

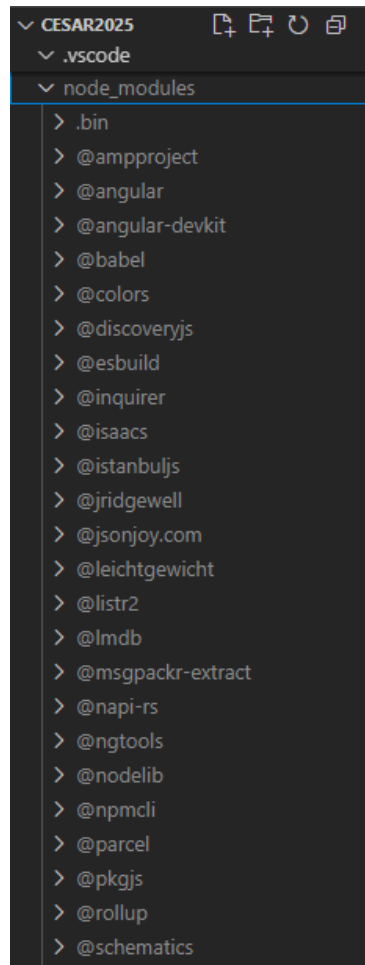
Contiene información de configuración de los editores de código



The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the project structure for 'CESAR2025'. The files listed are: .angular, .vscode (containing extensions.json, launch.json, and tasks.json), node_modules, public, src (containing app, index.html, main.server.ts, main.ts, server.ts, and styles.css), .editorconfig (selected), .gitignore, angular.json, package-lock.json, and package.json. On the right, the editor displays the content of the .editorconfig file, which is a configuration file for text editors. The file contains settings for the root directory and for specific file types like *.ts and *.md.

```
1 # Editor configuration, see https://editorconfig.org
2 root = true
3
4 [*]
5 charset = utf-8
6 indent_style = space
7 indent_size = 2
8 insert_final_newline = true
9 trim_trailing_whitespace = true
10
11 [*.ts]
12 quote_type = single
13 ij_typescript_use_double_quotes = false
14
15 [*.md]
16 max_line_length = off
17 trim_trailing_whitespace = false
18
```

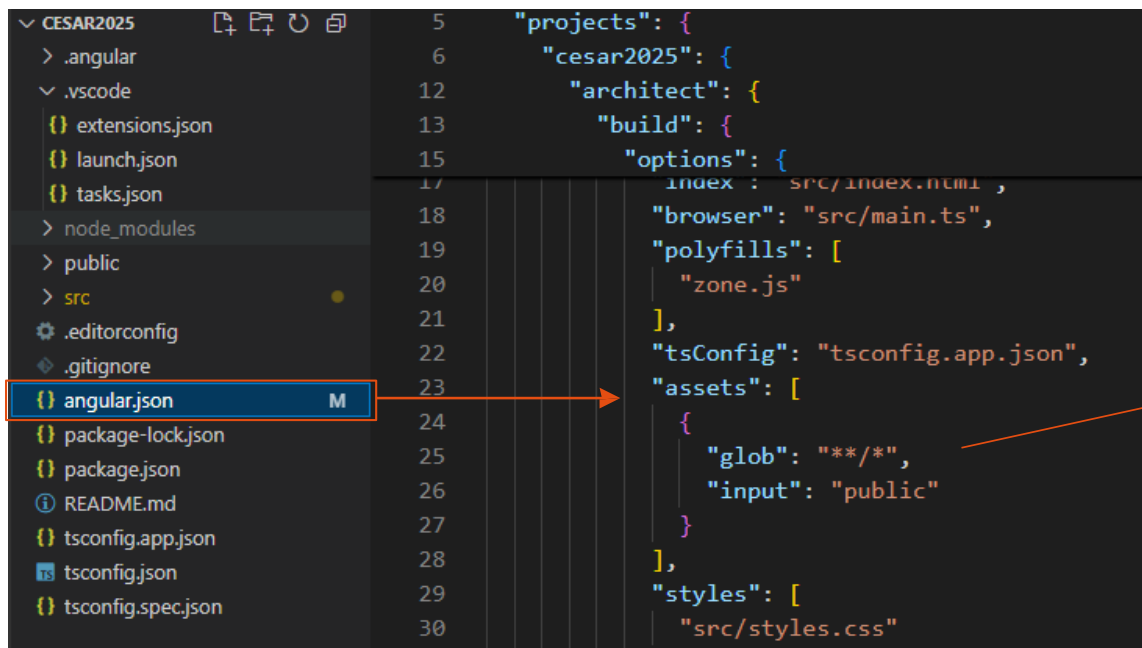
Estructura de un proyecto Angular



- **node_modules:** proporciona paquetes npm a todo el workspace

Estructura de un proyecto Angular

Proporciona valores predeterminados de configuración para el *workspace* y el proyecto, para las herramientas de compilación y desarrollo proporcionadas por la CLI de Angular



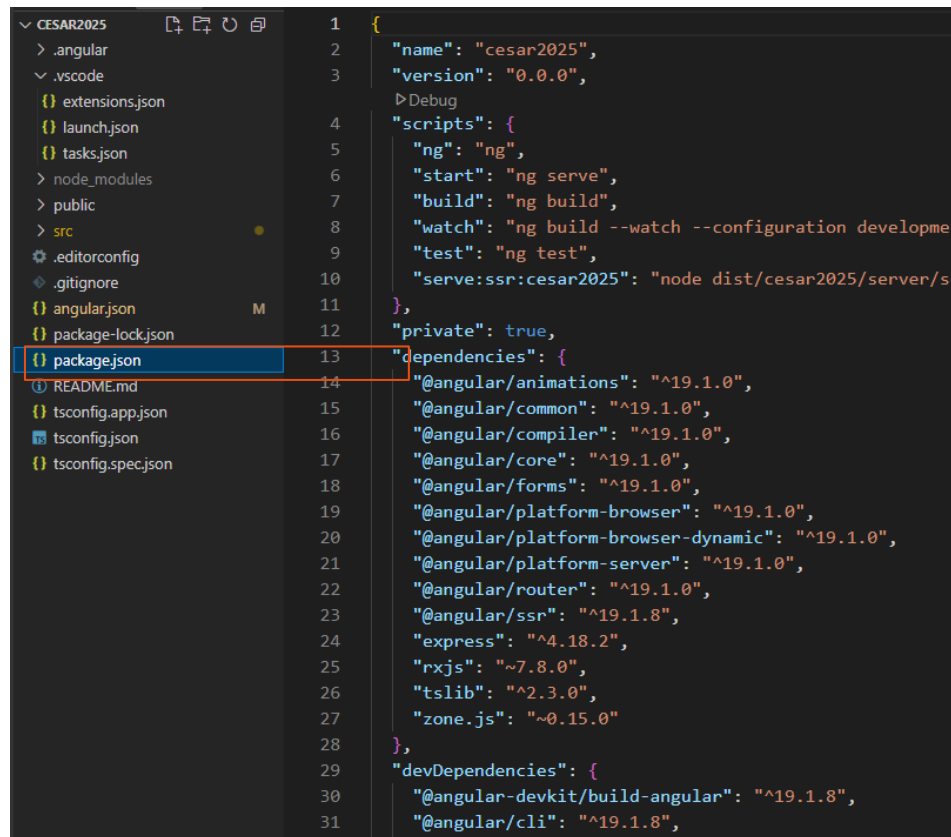
The screenshot shows the VS Code interface with a file explorer on the left and a code editor on the right. The file explorer shows the project structure for 'CESAR2025', including folders like '.angular', '.vscode', 'node_modules', 'public', and 'src', and files like 'angular.json', 'package-lock.json', 'package.json', 'README.md', 'tsconfig.app.json', 'tsconfig.json', and 'tsconfig.spec.json'. The 'angular.json' file is selected and highlighted with a red box. An orange arrow points from this box to the 'assets' section in the code editor. The code editor shows the 'angular.json' configuration, which includes settings for the 'cesar2025' project, such as 'architect', 'build', 'options', 'polyfills', 'tsConfig', 'assets', and 'styles'. The 'assets' section is highlighted with a red box, and an orange arrow points from it to the text on the right.

```
5  "projects": {
6    "cesar2025": {
12      "architect": {
13        "build": {
15          "options": {
17            "index": "src/index.html",
18            "browser": "src/main.ts",
19            "polyfills": [
20              "zone.js"
21            ],
22            "tsConfig": "tsconfig.app.json",
23            "assets": [
24              {
25                "glob": "**/*",
26                "input": "public"
27              }
28            ],
29            "styles": [
30              "src/styles.css"
```

Los recursos los tendremos que poner en una carpeta "assets" dentro de "public"

Estructura de un proyecto Angular

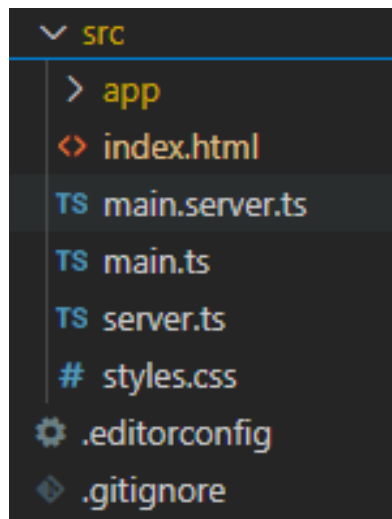
Configura las dependencias de los paquetes npm disponibles en los proyectos del *workspace*



```
1 {
2   "name": "cesar2025",
3   "version": "0.0.0",
4   "scripts": {
5     "ng": "ng",
6     "start": "ng serve",
7     "build": "ng build",
8     "watch": "ng build --watch --configuration development",
9     "test": "ng test",
10    "serve:ssr:cesar2025": "node dist/cesar2025/server/server.mjs",
11  },
12  "private": true,
13  "dependencies": {
14    "@angular/animations": "^19.1.0",
15    "@angular/common": "^19.1.0",
16    "@angular/compiler": "^19.1.0",
17    "@angular/core": "^19.1.0",
18    "@angular/forms": "^19.1.0",
19    "@angular/platform-browser": "^19.1.0",
20    "@angular/platform-browser-dynamic": "^19.1.0",
21    "@angular/platform-server": "^19.1.0",
22    "@angular/router": "^19.1.0",
23    "@angular/ssr": "^19.1.8",
24    "express": "^4.18.2",
25    "rxjs": "~7.8.0",
26    "tslib": "^2.3.0",
27    "zone.js": "~0.15.0"
28  },
29  "devDependencies": {
30    "@angular-devkit/build-angular": "^19.1.8",
31    "@angular/cli": "^19.1.8",
```



Estructura de un proyecto Angular



Carpeta principal donde desarrollaremos nuestro código

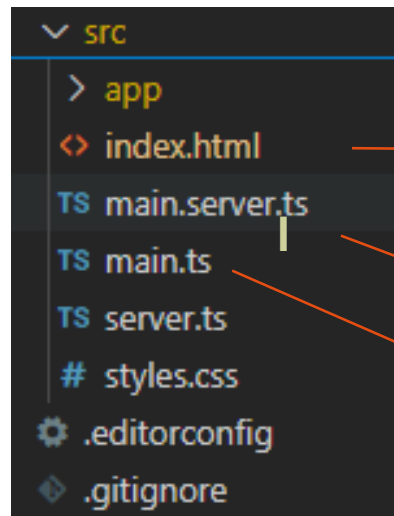
Carpeta donde se encuentra la app principal, los componentes, los modelos...

Página HTML principal. Por defecto carga `<app-root>`

Punto de entrada a la aplicación. Carga el componente principal en el navegador

Fichero de estilos global

Estructura de un proyecto Angular



```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Viu</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

```
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { config } from './app/app.config.server';

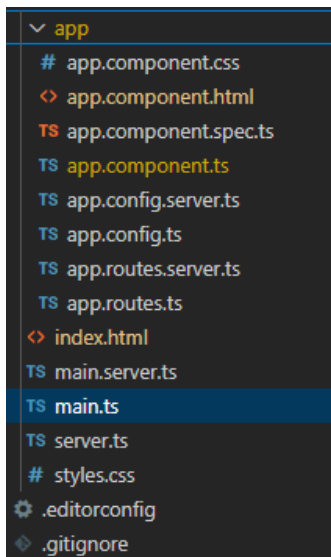
const bootstrap = () => bootstrapApplication(AppComponent, config);

export default bootstrap;
```

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

Estructura de un proyecto Angular

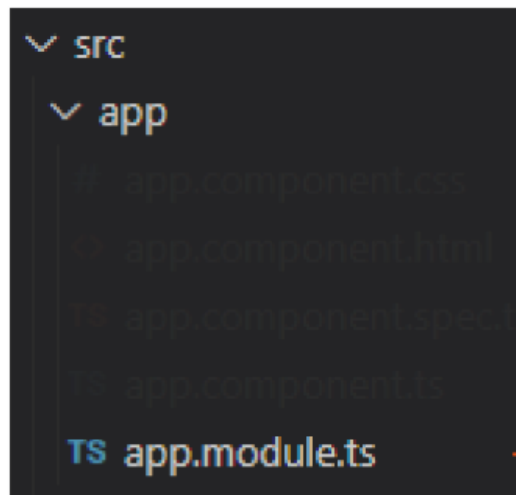


- Fichero de estilos del componente principal (por defecto, vacío)
- Página HTML que se cargará inicialmente
- Fichero para hacer pruebas
- Fichero TypeScript: →
- Configuración de aplicación
- Configuración de servidor (SSR)
- Fichero para gestionar routing

```
src > app > TS app.component.ts > ...
1  import { Component } from '@angular/co
2  import { RouterOutlet } from '@angular
3  import { InicioComponent } from './ini
4  import { CabeceraComponent } from '../
5  import { NavegacionComponent } from '.
6  import { PieComponent } from '../..co
7
8  @Component({
9    selector: 'app-root',
10   imports: [RouterOutlet, InicioCompon
11   templateUrl: './app.component.html',
12   styleUrls: ['./app.component.css']
13 })
14 export class AppComponent {
15   title = 'CesarMusic';
16 }
```

Estructura de un proyecto Angular

- Las aplicaciones no standalone tiene un módulo principal, que contiene todas las dependencias de los componentes que contienen
- Este módulo principal se creaba por defecto hasta Angular 17



Configuración del
módulo principal

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

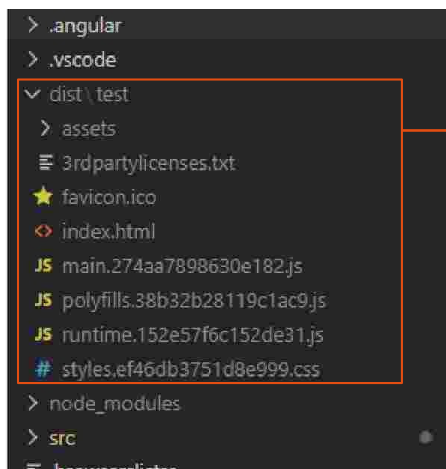
import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Despliegue de una aplicación Angular

- Compilando nuestra aplicación desde consola:

```
>> ng build
```



Al desplegar la aplicación nos crea una nueva carpeta dist

Aparte de compilar, hace el *minify uglify* → reducción del tamaño del código, quita los espacios en blanco y acorta el nombre de las variables

```
!==(n=this._teardowns)&&void 0!==(n?n:[]).push(t))}_hasParent(t){const[_  
this;this._parentage=Array.isArray(n)?(n.push(t),n):n?[_n,t]:t}_removePa  
downs:n}=this;p&&Qr(n,t),t instanceof it&&t._removeParent(this)}it.EMP  
e&&"closed" in e&&te(e.remove)&&te(e.add)&&te(e.unsubscribe)}function kc  
useDeprecatedSynchronousErrorHandling:!!useDeprecatedNextContext:!!},r  
out(e){const{delegate:t}=r;return((null==t?void 0:t.clearTimeout)||cle  
e;t(e))}}function io(){const Fv=Qs("C",void 0,void 0);function Qs(e,t  
nchronousErrorHandling){const t=!bn;if(t&&(bn={errorThrown:!!error:nul  
t){super(),this.isStopped=!1,t?(this.destination=t,0c(t)&&t.add(this)):  
{return Qs("N",e,void 0)}(t),this):this._next(t)}error(t){this.isStoppe  
.isStopped?Js(Fv,this):(this.isStopped=!0,this._complete())}unsubscribe  
t(t)}_error(t){try{this.destination.error(t)}finally{this.unsubscribe()  
t,n,r){let i;if(super(),te(t))i=t;else if(t){let o;({next:i,error:n,com  
o=t,i=null==i?void 0:i.bind(o),n=null==n?void 0:n.bind(o),r=null==r?void
```



Índice

- Introducción
- Contexto
- Funcionalidades básicas
 - Componentes
 - Declaración de propiedades
 - Binding por interpolación
 - Llamadas a eventos
- Routing
- Directivas, control de flujo y tuberías
- Modelos
- Paso de parámetros
- Servicios y peticiones HTTP
- Formularios
- Otros

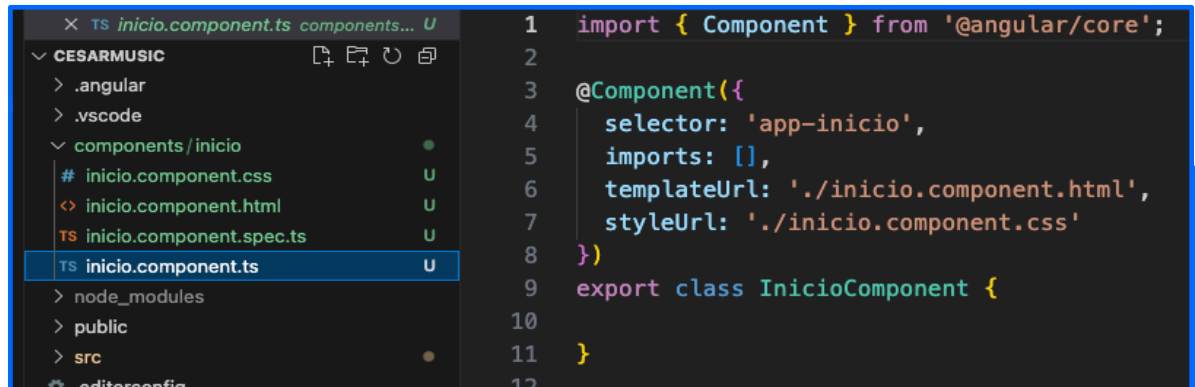
Componentes

- Creando componentes desde consola:

```
>> ng g component nombre_componente
```

```
rhhd@MacBook-Pro-de-Raquel components % ng g component inicio
Node.js version v23.3.0 detected.
Odd numbered Node.js versions will not enter LTS status and should not be used for production.
CREATE components/inicio/inicio.component.css (0 bytes)
CREATE components/inicio/inicio.component.html (21 bytes)
CREATE components/inicio/inicio.component.spec.ts (592 bytes)
CREATE components/inicio/inicio.component.ts (214 bytes)
```

- Crea 4 ficheros:
 - Fichero HTML
 - Fichero CSS
 - Fichero TS
 - Fichero de pruebas



The screenshot shows the VS Code interface. On the left, the file explorer displays the project structure: CESARMUSIC, .angular, .vscode, components/inicio, and node_modules. The file 'inicio.component.ts' is selected. On the right, the editor shows the content of 'inicio.component.ts'.

```
1 import { Component } from '@angular/core';
2
3 @Component({
4   selector: 'app-inicio',
5   imports: [],
6   templateUrl: './inicio.component.html',
7   styleUrls: ['./inicio.component.css']
8 })
9 export class InicioComponent {
10
11 }
12
```

Componentes

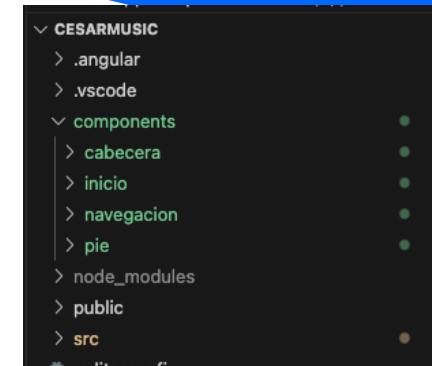
Estructura de Modelo Básico que vamos a crear



Componentes

- Creando componentes desde consola:

```
>> ng g component components/inicio
```

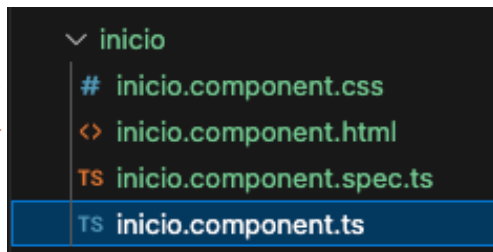


- Actualiza el fichero app.component.ts para añadirle el nuevo elemento creado

```
TS app.component.ts ●
src > app > TS app.component.ts > ...
1  import { Component } from '@angular/core';
2  import { CommonModule } from '@angular/common';
3  import { RouterOutlet } from '@angular/router';
4  import { InicioComponent } from './components/inicio/inicio.component';
5
6  @Component({
7    selector: 'app-root',
8    standalone: true,
9    imports: [CommonModule, RouterOutlet, InicioComponent],
10   templateUrl: './app.component.html',
11   styleUrls: ['./app.component.css']
12 })
13 export class AppComponent {
14   title = 'viu2024';
15 }
```

Componentes

- Ficheros del componente creado:



Vacío

Utilizado para hacer tests



Cuando invoquemos al componente (por ejemplo, en app.component.html) utilizaremos esta etiqueta:

`<app-inicio></app-inicio>`

Importación de otros componentes

Clase, la cual puede contener propiedades y métodos

Constructor



Ciclo de vida de los componentes (hooks)

Método	Descripción
ngOnInit	Se invoca al iniciar la directiva o componente y se han recibido las propiedades de entrada
ngDoCheck	Se invoca cuando hay cambios en la directiva o componente
ngOnChanges	Se invoca al inicio y cada vez que Angular detecta un cambio en los inputs del componente
ngOnDestroy	Se invoca cuando Angular destruye la directiva o componente
...	

Método	Descripción
afterRender	Se invoca cada vez que la aplicación acaba un renderizado
afterNextRender	Se invoca la próxima vez que la aplicación finalice un renderizado

Desde Angular 16

<https://angular.dev/style-guide#lifecycle-hooks>

Componentes

- Declaración de variables

```
export class InicioComponent implements OnInit {  
  ngOnInit(): void {  
    throw new Error('Method not implemented.');  }  
  public titulo:number = "MusicCesar";  
  public pulsaciones:number = 0;  
}
```

.ts

Variable mal declarada → error de tipado detectado

Variable bien declarada

- Es una buena práctica inicializar el valor de las variables en el constructor

Interpolación

- **Interpolación:** permite visualizar de forma sencilla propiedades de un componente en su plantilla (*template*)

```
export class InicioComponent {  
  public titulo:string;  
  constructor() {  
    this.titulo = "César Music"  
  }  
}
```

.ts

```
1  
2 <h1>Bienvenido a la página de Inicio de {{titulo}} </h1>  
3
```

.html

```
export class MusicaComponent {  
  public record:any;  
  constructor() {  
    this.record = {title: "To the faithful departed", artist: "The Cranberries", year: 1996};  
  }  
}
```

```
<h2>DISCO</h2>  
<p>{{record.title}}</p>  
<p>{{record.artist}}</p>  
<p>{{record.year}}</p>
```

Interpolación y uso de funciones

.ts

```
export class InicioComponent {  
  public titulo:string;  
  public pulsaciones:number;  
  
  constructor(){  
    this.titulo = "CesarMusic";  
    this.pulsaciones = 0;  
  }  
  pulsar():void{  
    this.pulsaciones++;  
  }  
}
```

.html

```
<h1>Bienvenido a la página Musical del César - {{titulo}}</h1>  
<!-- Declaración de la función de Pulsaciones -->  
<button (click)="pulsar()">Púlsame</button> |  
<p>Has pulsado el botón {{pulsaciones}} veces</p>
```

Breve introducción al control de flujo en Angular

.ts

```
export class InicioComponent {  
  public titulo:string;  
  public pulsaciones:number;  
  
  constructor(){  
    this.titulo = "CesarMusic";  
    this.pulsaciones = 0;  
  }  
  
  pulsar():void{  
    this.pulsaciones++;  
  }  
  
  reset():void{  
    this.pulsaciones=0;  
  }  
}
```

.html

```
@if(pulsaciones > 0) {  
  <p>Has pulsado el botón {{pulsaciones}} veces</p>  
  <button (click)="reset()">Borrado</button>  
}
```

Lo que hay entre llaves solo se mostrará por pantalla cuando se cumpla la condición introducida



Índice

- Introducción
- Contexto
- Funcionalidades básicas
- Routing
- Directivas, control de flujo y tuberías
- Modelos
- Paso de parámetros
- Servicios y peticiones HTTP
- Formularios
- Otros



Sobre la navegación web

- La navegación web se lleva a cabo a través de una serie de acciones:
 - Al introducir una URL en el navegador se carga una página
 - Al seleccionar un elemento navegable (vínculo) de una página se cambia de página
 - Al darle a los botones de atrás y adelante del navegador se cambia de página
- Las aplicaciones web se adaptan a dicha forma de navegación utilizando un sistema de gestión de las rutas y las vistas asociadas
 - En Angular dicha gestión se realiza con el servicio **Angular Router**



Routing

- Paso 1: Modificación del fichero de rutas (app.routes.ts)
 - Importación de los módulos del router de Angular y de los componentes a utilizar

```
import { Routes } from '@angular/router';
import { CineComponent } from '../components/cine/cine.component';
import { InicioComponent } from '../components/inicio/inicio.component';
import { MusicaComponent } from '../components/musica/musica.component';
```

app.routes.ts

- Definición del array de rutas

```
export const routes: Routes = [
  {path: 'inicio', component: InicioComponent},
  {path: 'musica', component: MusicaComponent},
  {path: 'cine', component: CineComponent}
];
```

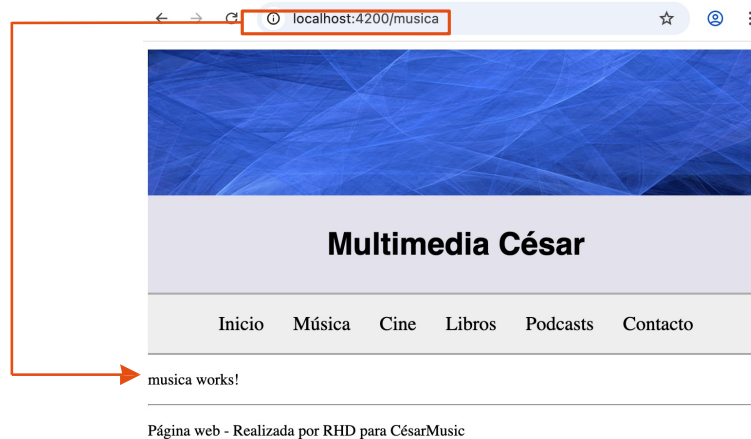
Lista de las rutas disponibles
en la aplicación

Routing

- Paso 2: Utilización de `<router-outlet>` en `app.component.html`:

```
<router-outlet></router-outlet>
```

Ahora cuando en la barra de navegación pongamos `url/componente` cargará al componente indicado



Routing: refinando las rutas

```
const appRoutes: Routes = [  
  {path: '', component: InicioComponent},  
  {path: 'inicio', component: InicioComponent},  
  {path: 'musica', component: MusicaComponent},  
  {path: 'cine', component: CineComponent},  
  {path: '**', component: ErrorComponent}  
];
```

app.routes.ts

Indicamos el componente a cargar cuando no se indica parámetros en la ruta

Creamos un componente “Error” que se carga cuando se le pasan parámetros no definidos

NOTA: esta declaración debe hacerse siempre al final

Routing: paso de parámetros

app.routes.ts

```
const appRoutes: Routes = [
  { path: '', component: InicioComponent },
  { path: 'inicio', component: InicioComponent },
  { path: 'musica', component: MusicaComponent },
  { path: 'cine', component: CineComponent },
  { path: 'cine/:genero', component: CineComponent },
  { path: '**', component: ErrorComponent }
];
```

Paso 1: indicar el/los parámetro(s) a pasar

Paso 2: importar ActivatedRoute y Params

Paso 3: declarar la variable 'route' de tipo ActivatedRoute

Paso 4: suscribirse a 'route' (subscribe) y extraer el parámetro indicado en la URL

cine.component.ts

```
import { Component, afterNextRender } from '@angular/core';
import { ActivatedRoute, Params } from '@angular/router';

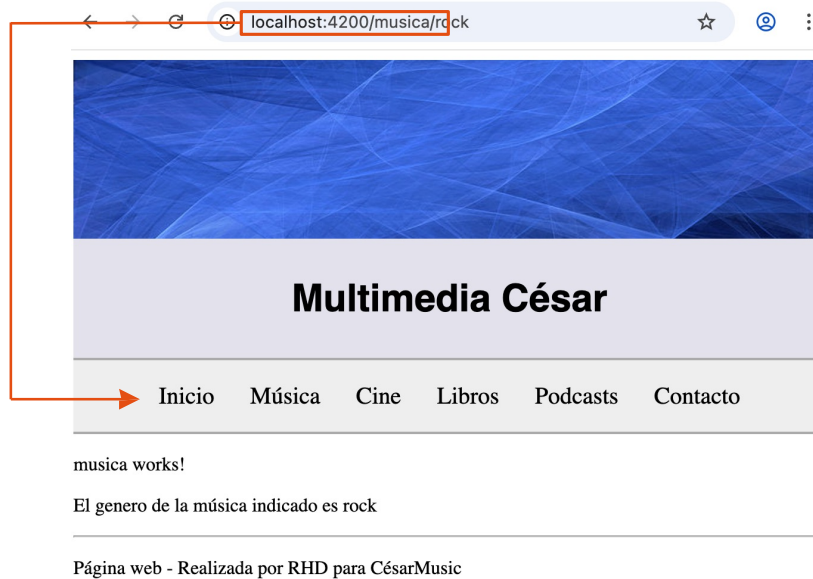
@Component({
  selector: 'app-cine',
  standalone: true,
  imports: [],
  templateUrl: './cine.component.html',
  styleUrls: ['./cine.component.css']
})
export class CineComponent {
  public genero:string="";

  constructor(
    private route:ActivatedRoute
  ){
    afterNextRender(()=>{
      this.route.params.subscribe((params: Params) =>{
        this.genero = params["genero"];
      });
    });
  }
}
```

Routing: paso de parámetros

cine.component.html

```
<p>cine works!</p>
@if (genero) {
  <p>Este es el género: {{genero}}</p>
}
```



RouterLink

- routerLink permite cargar diferentes componentes desde la barra de navegación. Se utiliza en los elementos ancla <a>
 - Imaginemos que tenemos un componente “navegacion.component” encargado de gestionar la barra de navegación:

```
<nav class="menu_principal">
  <ul>
    <li><a [routerLink]="['/']">Inicio</a></li>
    <li><a [routerLink]="['/musica']">Música</a></li>
    <li><a [routerLink]="['/cine']">Cine</a></li>
    <li><a [routerLink]="['/libros']">Libros</a></li>
    <li><a [routerLink]="['/podcasts']">Podcasts</a></li>
    <li><a [routerLink]="['/videojuegos']">Videojuegos</a></li>
  </ul>
</nav>
```

navegacion.component.html

Tendrá el mismo comportamiento que si escribimos en la barra del navegador: url/cine

Funciona si previamente hemos configurado las rutas en app.routes.ts

→ En el fichero “navegacion.component.ts” habrá que importar RouterLink

routerLinkActive

- routerLinkActive permite añadir una clase CSS cuando una ruta está activa

```
<nav class="menu_principal">
  <ul>
    <li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/']">Inicio</a></li><!--
    --><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/musica']">Música</a></li><!--
    --><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/cine']">Cine</a></li><!--
    --><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/libros']">Libros</a></li><!--
    --><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/podcasts']">Podcasts</a></li><!--
    --><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/videojuegos']">Videojuegos</a></li>
  </ul>
</nav>
```

navegacion.component.html

Se le asignará la clase de CSS
"seleccionado" cuando dicha ruta está
activa

```
nav.menu_principal ul > li.seleccionado {
  background-color: #e65218;
}
```

navegacion.component.css

- En el fichero "navegacion.component.ts" habrá que importar RouterLinkActive

RouterLinkActive

- routerLinkActive tiene una propiedad routerLinkActiveOptions que comprueba si la ruta es exactamente la indicada (y evitar que haya activos 2 enlaces)

```
<nav class="menu_principal">
  <ul>
    <li [routerLinkActive]="['seleccionado']" [routerLinkActiveOptions]="{exact: true}"><a [routerLink]="['/']">Inicio</a></li><!--
--><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/musica']">Música</a></li><!--
--><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/cine']">Cine</a></li><!--
--><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/libros']">Libros</a></li><!--
--><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/podcasts']">Podcasts</a></li><!--
--><li [routerLinkActive]="['seleccionado']"><a [routerLink]="['/videojuegos']">Videojuegos</a></li>
  </ul>
</nav>
```

navegacion.component.html

Así evitamos que se marque la ruta raíz (/) cuando seleccionamos otra ruta

Router

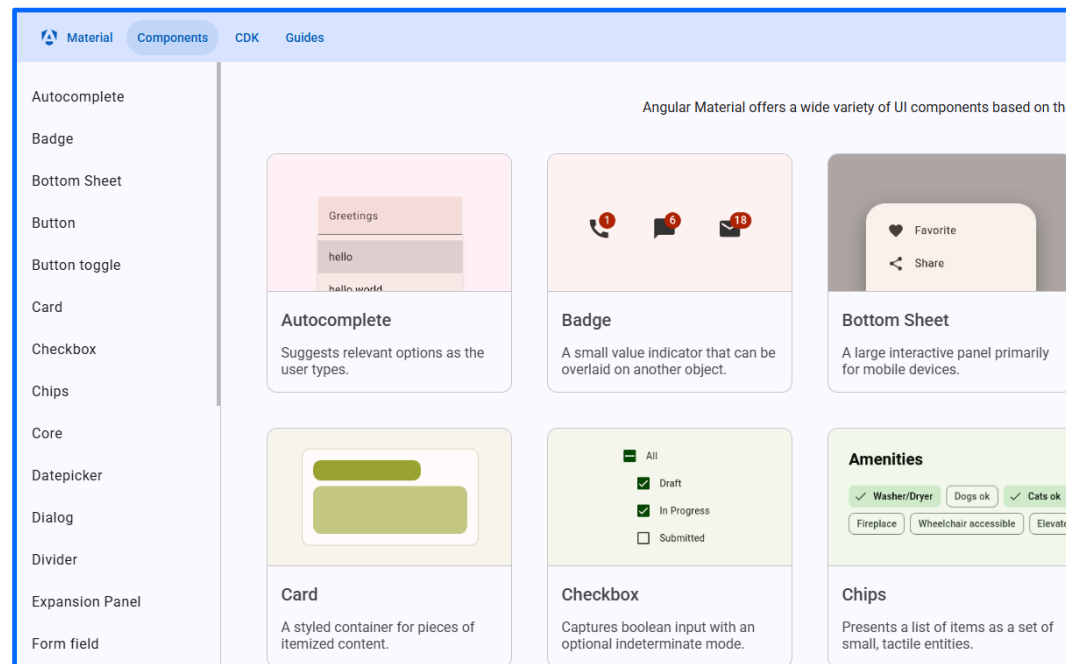
- router es un servicio que proporciona navegación entre las diferentes vistas
 - Si en el ejemplo anterior queremos ir a una ruta desde TypeScript en vez de utilizando elementos HTML podemos usar el método `router.navigate()`

```
private router:Router  
  
navegacion(ruta:string){  
  | this.router.navigate(['/'+ruta]);  
}
```

→ Desde un método estamos navegando hasta una vista

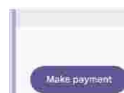
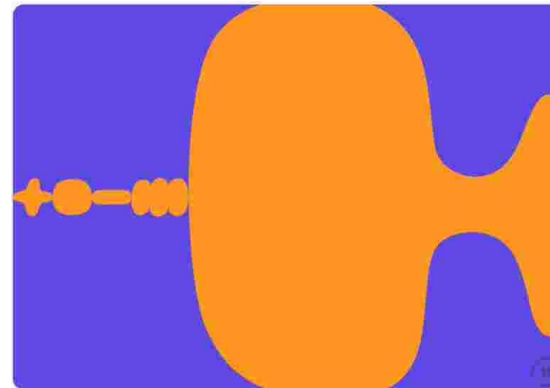
Angular Material

- Acceso a múltiples componentes para mejorar la funcionalidad y aspecto de las aplicaciones web basado en la especificación de Material Design
- Enlace: <https://material.angular.io/>



Material 3

- Última versión del sistema de diseño de código abierto de Google
- Enlace: <https://m3.material.io/>





Bibliografía

- Uluca, D. (2018). Angular 6 for Enterprise-Ready Web Applications: Deliver Production-Ready and Cloud-scale Angular Web Apps. Packt Publishing, Limited
- Monteiro, F. (2018) Hands-On Full Stack Web Development with Angular 6 and Laravel 5: Become Fluent in Both Frontend and Backend Web Development with Docker, Angular and Laravel. Packt Publishing, Limited
- Boada, M., Gómez, J. A. (2018). El gran libro de Angular. Marcombo
- Angular página web. HTML. Disponible en línea: <https://angular.dev/>